

C++之STL容器详解（包含十种常用容器）

https://blog.csdn.net/2203_75327677/article/details/136080887

文章目录

前言

一、string 类

二、vector 容器

三、set 容器

四、queue 容器

五、priority_queue 容器

六、stack 容器

七、deque 容器

八、map 容器

九、pair 容器

十、bitset 容器

前言

C++的STL是**提高编写效率**的一个利器，是**算法类竞赛**必不可少的工具，本篇博客将详细总结C++**STL的各种容器**及其**常用方法**，包括：

- string 类
- vector 容器
- set 容器
- queue 容器
- priority_queue 容器
- stack 容器
- deque 容器
- map 容器
- pair 容器
- bitset 容器

一、string 类

string类本不是STL的容器，但是它与 **STL容器** 有着很多相似的操作，因此，把string放在这里一起进行介绍。C++中的string类相当于是字符数组，但是其更方便于我们的操作，而且不需要在输入之前初始化字符数组的容量，节省空间。

```
1  #include<string>
2
3  /*string 生成*/
4  string str("123"); //"123"
5  string str(3,"1"); //"111"
6  string str = "123"; //"123"
7
8  /*string 头尾*/
9  str.front(); //第一个元素
10 str.back(); //最后一个元素
11
12 /*string 迭代器*/
13 string::iterator iter //一个string的迭代器
14 str.begin() //指向str第一个元素位置的迭代器
15 str.end() //指向str最后一个元素后一个位置的迭代器
16
17 /*string 插入*/
18 str.push_back('a'); //在尾部插入一个字符
19 str.insert(s1.begin(), 'a'); //在指定位置前面插入一个字符
20
21 /*string 删除*/
22 str.pop_back(); //删除最后一个元素
23 str.erase(str.begin()); //删除迭代器指向元素
24 str.erase(str.begin()+1, str.end()-2); //删除指定区间的元素
25 str.erase(1, 6); //删除从索引（1）开始的n（6）个字符
26
27 /*string 替换*/
28 str.replace(str.begin(), str.begin + 5, "boy"); //替换迭代器指定的区间
29 str.replace(6, 5, "girl"); //替换索引指定的区间，从下标6开始的五个元素
30
31 /*string 拼接*/
32 str1.append(str2); //str1str2
33 str1 = str1 + str2; //str1str2
34
35 /*string 容量*/
36 str.size()
37 str.length()
38
39 /*string 遍历*/
```

```

40 for(int i = 0; i < str.size(); i ++ ) //索引遍历
41 for(string::iterator iter = str.begin(); iter != str.end(); iter ++ )
42 for(auto &x : str) //迭代器另一种便捷方法
43
44 /*string 排序*/
45 sort(str.begin(), str.end());
46
47 /*string 比较*/
48 str1 < str2 //字典序比较, <、<=、==、>、>=都可以用
49 str1.compare(str2) //相等为0, str1>str2为1, 反之-1
50
51 /*string 查找*/
52 str.find("123") //从前往后找, 若找到, 返回首字母下标; 反之, 返回-1
53 str.rfind("123") //从后往前找
54 str.find_first_of("123") //查找第一个属于该字符串的字符返回下标
55 str.find_first_not_of("123")
56 str.find_last_of("123")
57 str.find_last_not_of("123")
58
59 /*string 某元素个数*/
60 str.count('a'); //返回str里a字符的个数
61
62 /*string 分割*/
63 str.substr(2, 5); //返回从索引2开始的五个元素组成的字符串
64
65 /*string 判空*/
66 str.empty() //返回布尔值
67
68 /*string 清空*/
69 str.clear();

```

二、vector 容器

vector是变长数组（节省空间），支持随机访问，不支持在任意位置 $O(1)$ 插入。为了保证效率，元素的增删一般应该在末尾进行。（以下typename均用int举例）

```

1 #include <vector>
2
3 /*vector 生成*/
4 vector<int> vct(3); //0 0 0
5 vector<int> vct(3, 5); //5 5 5
6 vector<int> vct{1, 2, 3}; //1 2 3
7
8 /*vector 头尾*/

```

```

9  vct.front(); //第一个元素
10 vct.back(); //最后一个元素
11
12 /*vector 迭代器*/
13 vector<int>::iterator iter //一个vector的迭代器
14 vct.begin() //指向vct第一个元素位置的迭代器
15 vct.end() //指向vct最后一个元素后一个位置的迭代器
16
17 /*vector 插入*/
18 vct.push_back(2); //在尾部插入一个2
19 vct.insert(vct.begin(), 2); //在指定位置前面插入一个元素
20
21 /*vector 删除*/
22 vct.pop_back();
23 vct.erase(vct.begin());
24 vct.erase(vct.begin()+1, vct.end()-2);
25 vct.erase(1, 6);
26
27 /*vector 容量*/
28 vct.size()
29
30 /*vector 遍历*/
31 for(int i = 0; i < vct.size(); i ++ ) //索引遍历
32 for(vector<int>::iterator iter = vct.begin(); iter != vct.end(); iter
33 for(auto &x : vct) //迭代器另一种便捷方法
34
35 /*vector 排序*/
36 sort(vct.begin(), vct.end());
37
38 /*vector 查找*/
39 vct.find(2) //从前往后找，若找到，返回指向该处的迭代器；反之，返回迭代器vct.end()
40
41 /*vector 某元素个数*/
42 vct.count(2); //返回容器里2的个数
43
44 /*vector 判空*/
45 vct.empty() //返回布尔值
46
47 /*vector 清空*/
48 vct.clear();

```

三、set 容器

头文件set主要包括set和multiset（两者方法相同，所以以下只用set举例）两个容器，分别是“有序集合”和“有序多重集合”，即前者的元素不能重复，而后者可以包含若干个

相等的元素，两者都会将其容器内的元素从小到大自动排序。set和multiset的内部实现是一棵红黑树，它们支持的函数基本相同。（下typename均用int举例）

```
1  #include <set>
2
3  /*set 生成*/
4  set<int> st;
5
6  /*set 迭代器*/
7  set<int>::iterator iter
8  st.begin()
9  st.end()
10
11 /*set 插入*/
12 st.insert(2); //插入一个元素
13
14 /*set 删除*/
15 st.erase(st.begin()); //删除迭代器指向元素
16 st.erase(2); //删除所有为2的元素
17
18 /*set 容量*/
19 st.size()
20
21 /*set 查找*/
22 st.find(2) //从前往后找，若找到，返回指向该处的迭代器；反之，返回迭代器st.end()
23 st.lower_bound(x) //二分查找大于等于x的元素中最小的一个，并返回指向该元素的迭代器
24 st.upper_bound(x) //二分查找大于x的元素中最小的一个，并返回指向该元素的迭代器。
25
26 /*set 某元素个数*/
27 st.count(2); //返回容器里2的个数
28
29 /*set 判空*/
30 st.empty() //返回布尔值
31
32 /*set 清空*/
33 st.clear();
```

四、queue 容器

头文件queue主要包括循环队列queue和优先队列priority_queue两个容器。其中queue容器相当于队列，满足先进先出的原则，即队尾出、队首进。（以下typename均用int举例）

```

1  #include <queue>
2
3  /*queue 生成*/
4  queue<int> q;
5
6  /*queue 头尾*/
7  q.front();
8  q.back();
9
10 /*queue 插入*/
11 q.push(2); //在队尾插入一个元素
12
13 /*queue 删除*/
14 q.pop(); //在队首删除一个元素
15
16 /*queue 容量*/
17 q.size();
18
19 /*queue 判空*/
20 q.empty()

```

五、priority_queue 容器

头文件queue主要包括循环队列queue和优先队列priority_queue两个容器。其中priority_queue容器相当于大根堆（或者小根堆），大根堆每次堆顶是最大元素，小根堆每次堆顶是最小元素。（以下typename均用int举例）

```

1  #include <queue>
2
3  /*priority_queue 生成*/
4  priority_queue<int> q; //大根堆
5  priority_queue<int, vector<int>, greater<int>> q; //小根堆
6
7  /*priority_queue 插入*/
8  q.push(2); //把一个元素插入堆
9
10 /*priority_queue 删除*/
11 q.pop(); //删除堆顶的元素
12
13 /*priority_queue 堆顶*/
14 q.top(); //返回堆顶元素
15
16 /*priority_queue 容量*/
17 q.size();

```

```
18
19  /*priority_queue 判空*/
20  q.empty()
```

六、stack 容器

stack容器相当于栈，满足先进后出的原则，即插入和删除都只能在栈顶操作。（以下typename均用int举例）

```
1  #include <stack>
2
3  /*stack 生成*/
4  stack<int> sk;
5
6  /*stack 插入*/
7  sk.push(2); //把一个元素放入栈
8
9  /*stack 删除*/
10 sk.pop(); //删除栈顶的元素
11
12 /*stack 栈顶*/
13 sk.top(); //返回栈顶元素
14
15 /*stack 容量*/
16 sk.size();
17
18 /*stack 判空*/
19 sk.empty()
```

七、deque 容器

双端队列deque是一个支持在两端高效插入或删除元素的连续线性存储空间。它就像是vector和queue的结合。与vector相比，deque在头部增删元素仅需要 $O(1)$ 的时间；与queue相比，deque像数组一样支持随机访问。（以下typename均用int举例）

```
1  #include <deque>
2
3  /*deque 生成*/
4  deque<int> dq;
5
6  /*deque 头尾*/
7  dq.front();
```

```

8 dq.back();
9
10 /*dequeue 迭代器*/
11 dq.begin()
12 dq.end()
13
14 /*dequeue 插入*/
15 dq.push_front(2); //头插入
16 dq.push_back(2); //尾插入
17
18 /*dequeue 删除*/
19 dq.pop_front(); //头删除
20 dq.pop_back(); //尾删除
21
22 /*dequeue 容量*/
23 dq.size();
24
25 /*dequeue 判空*/
26 dq.empty()
27
28 /*dequeue 清空*/
29 dq.clear();

```

八、map 容器

map容器是一个键值对key-value的映射，其内部实现是一棵以key为关键码的红黑树。map的key和value可以是任意类型。（以下typename均用int举例）

```

1 #include <map>
2
3 /*map 生成*/
4 map<key_type, value_type> name;
5 map<int, int> mp;
6
7 /*map 迭代器*/
8 map<int, int>::iterator iter
9 mp.begin()
10 mp.end()
11
12 /*map 键值*/
13 iter->first //key
14 iter->second //value
15
16 /*map 插入*/

```



```

17 mp[2] = 5; //直接添加
18 mp.insert(pair<int, int>(2, 5)); //insert一个pair
19
20 /*删除*/
21 mp.erase(iter); //删除迭代器所指的键值对
22
23 /*map 容量*/
24 mp.size()
25
26 /*map 查找*/
27 mp.find(2) //从前往后找，若找到，返回指向该处的迭代器；反之，返回迭代器mp.end()
28
29 /*map 某元素个数*/
30 st.count(2); //返回key为2的个数（map中只可能是0或者1）
31
32 /*map 判空*/
33 mp.empty() //返回布尔值
34
35 /*map 清空*/
36 mp.clear();

```

九、pair 容器

pair相当于将两份数据打包成一对，两份数据的数据类型任意，多与其他容器混合使用，单独使用的情况比较少。（以下typename均用int举例）

```

1  #include <utility>
2
3  /*pair 生成*/
4  pair<int, int> pr = make_pair(0,1);
5  pair<int, int> pr(0, 1);
6
7  /*pair 两个值*/
8  pr.first
9  pr.second
10
11 /*pair 多与其他容器结合使用*/
12 set<pair<int, int>> st;
13 vector<pair<int, int>> vct(mp.begin(), mp.end());

```

十、bitset 容器

bitset容器相当于是01数组，其方便之处是可以直接按位（下标）进行位运算，但是要

注意下标从小到大依次存低位到高位，是逆序的。

```
1  #include<bitset>
2
3  /*bitset 生成*/
4  bitset<4> bt; //生成4位二进制数，初始化为0000
5  bitset<8> bt(12); //生成8位二进制数，且将10进制数12转化为2进制数存入其中
6  bitset<8> bt(str); //str可以是只有01的字符串或者字符数组
7
8  /*bitset 位运算相关*/
9  bt1 |= bt2; //两个二进制数取或操作
10 bt1 &= bt2; //两个二进制数取与操作
11 bt1 ^= bt2; //取异或
12 bt1 = ~bt1; //取反
13 bt1 <<= x; //左移右移
14
15 bt.test(x) //判断第x个位置是0还是1，也就是输出第x个位置，注意逆序
16
17 bt.flip(); //将二进制每一位取反
18 bt.flip(x); //将二进制第x位取反
19 bt.set(); //将二进制每一位位置为1
20 bt.set(x); //将第x个位置置为1
21 bt.reset(); //将二进制每一位位置为0
22 bt.reset(x); //将第x个位置置为0
23
24 /*bitset 容量*/
25 bt.size() //二进制数组的长度，就是定义的长度；
26
27 /*bitset 某元素个数*/
28 bt.count(); //查询二进制数组中，1的个数
29
30 /*bitset 转化字符串*/
31 string str = bt.to_string(); //将二进制数组转化为字符串。
32
```